Color codes
Reserved words
Forbidden field or entity names
Grouping Operators
Functions
Filters
Related entities filtering transversal
Returned first level fields
Returned multilevel fields
Related entities data expansion
Pagination
URL S STANDARDIZATION
Response Conventions for Entity Modification Endpoints
General
PATCH / 204
DELETE / 204
ANEXO
PROCESS MANAGER API
URL S STANDARDIZATION

Color codes		
Approved direct option	Approved alternative	Postponed

Reserved words
Starting with _

Forbidden field or entity names
Starting with _ (the only exception are attribute names, because there is no API endpoint that goes directly to that entity, and therefore, it always has the entity name first, being unambiguous. Ie. `attributes._<attribute_name>`)
Containing a .
Containing a [

Grouping Operators		
Logic	V3	Examples
`<filter1> AND <filter2>`	**<filter1>&<filter2>**	**GET /api/v3/products?sku=A1&status=Draft**
	_and[0]<filter1>&_and[0]<filter2>	**GET /api/v3/products?_and[0]sku=A1&_and[0]status=Draft**
`<filter1> OR <filter2>`	**_or[#]<filter1>&_or[#]<filter2>**	**GET /api/v3/products?_or[0]sku=A1&_or[0]label=Shirt**

`(<filter1> OR <filter2>)`
`AND`
`(<filter3> OR <filter4>)`
	_or[0]<filter1>&_or[0]<filter2>&
	_or[1]<filter3>&_or[1]<filter4>
		GET
		/api/v3/products?_or[0]sku=A1&_or[0]label=Shirt&_or[1]status=Draft&_or[1]num_variations[gt]=0
		Sibling groups are ANDed.

	_and[0]_or[0]<filter1>&_and[0]_or[0]<filter2>& _and[0]_or[1]<filter3>&_and[0]_or[1]<filter4>	**GET** **/api/v3/products?_and[0]_or[0]sku=A1&_and[0]_or[0]label=Shirt&_and[0]_or[1]status=Draft&_and[0]_or[1]num_variations[gt]=0**

`(<filter1> AND <filter2>)`
`OR`
`(<filter3> AND <filter4>)`
	_or[0]_and[0]<filter1>&_or[0]_and[0]<filter2>&
	_or[0]_and[1]<filter3>&_or[0]_and[1]<filter4>
		GET
		/api/v3/products?_or[0]_and[0]sku=A1&_or[0]_and[0]label=Shirt&_or[0]_and[1]status=Draft&_or[0]_and[1]num_variations[gt]=0

Grouping Operators

`NOT (<filter1> <bool> <filter2>)`	**<!bool>[0]<filter1>&<!bool>[0]<filter2>** **eg: _!or[0]<filter1>&_!or[0]<filter2>**	`GET /api/v3/products?_!or[0]sku=A1&_!or[0]label=Shirt`

`SELECT`
`  …`
`FROM`
`  t1 LEFT JOIN`
`  t2 ON`
`  t1.t2_fk = t2.id AND`
`  <t2_filter>`
`WHERE`
`  t2.id IS NULL`
 	
`AND EXISTS (SELECT * FROM T2 WHERE )`
	_!exists[#]<t2_filter>
	
	Por public APIs, this must be heavily restricted.
	
	For the public PIM API V3.0, it will only be allowed for relationships.id or relationships.label and without being grouped with _and or _or
		`GET /api/v3/products?_!exists[0]product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc`

`SELECT`
`  …`
`FROM`
`  t1 LEFT JOIN`
`  t2 ON`
`  t1.id = t2.t1_fk AND`
`  <t2_filter>`
`WHERE`
`  t2.id IS NULL`		

`(3+ level nesting)`	Allowed, although it is advisable to study where to limit the depth for each API.	`GET /api/v3/products?_or[0]_and[0]_or[0]sku=A1&_or[0]_and[0]_or[0]label=Shirt&_or[0]_and[0]_or[1]status=Draft`

Functions

General pattern (<field>[<function>] The 1st arg is the value of the field) (<entity_x:n>[<function>] The 1st arg is a list) (<entity_x:1>[<function>] The 1st arg is an instance of the entity) (<entity_x:n_ending_in_.>[<function>] The 1st arg is an instance of the entity)	**<field or entity>[<function>:<*arg1>:<*...>:<*argn>][<filter>]=<value>** (args are literals. If we want references to other fields or entities, it must be in a V3.<gt_0>)	`GET /api/v3/products?label[lower][length][gt]=3`
String or Array/List length	**<field or entity_x:n>[length][<filter>]=<value>** Eg. countries[length][gt]=1	`GET /api/v3/products?label[length][lte]=256`

Functions		

	Eg. description[length][lte]=256
	Eg. related_products[length][eq]=0
		`GET /api/v3/products?overwritten_attributes[length][gt]=1`
		
		`GET /api/v3/products?product_relationships[length][eq]=0`

Full days passed from the field's date to the current date.
 	
(Since the date fields are considered to be UTC, the current date is the UTC one too, unless a tz database code)
	<field>[days_passed:<*tz_database_name>][<filter>]
	
	Eg. creation_date[days_passed][gt]=7 (UTC is inferred)
	Eg. last_update[days_passed:Europe/Madrid][lt]=1
		`GET /api/v3/products?created[days_passed][gt]=7`
		
		`GET /api/v3/products?modified[days_passed:Europe/Madrid][lt]=1`

Custom filters (can be injected in each API)	Eg. .[last_updated_by][eq]=admin Eg. related_products.[last_updated_by][eq]=admin Eg. country_code[continent]=africa	

Filters		
Filters only apply to fields		`GET /api/v3/products?categories[exists]`

`Field = 'value'`	**field=value**
	(alt) field[eq]=value
		`GET /api/v3/products?sku=A1`
		
		`GET /api/v3/products?sku[eq]=A1`

`(Universal Group - equality filter)`
`field <> 'value' OR field IS NULL OR NOT EXISTS(field)`
	(If the field is undefinable)
	_or[#]field[!eq]=value&_or[#]field[!exists]
	
	(If the field is nullable)
	_or[#]field[!eq]=value&_or[#]field[!null]
	
	(If the field is undefinable and nullable)
	_or[#]field[!eq]=value&_or[#]field[!exists]&_or[#]field[!null]
		`GET /api/v3/products?_or[0]label[!eq]=Shirt&_or[0]label[!exists]`
		
		`GET /api/v3/products?_or[0]label[!eq]=Shirt&_or[0]label[null]`

`("not equal" among the elements with value)` `field <> 'value'`	**field[!eq]=value**	`GET /api/v3/products?sku[!eq]=A1`

Filters		
	(If the field is undefinable) field[!eq]=value&field[exists] (If the field is nullable) field[!eq]=value&field[!null] (If the field is undefinable and nullable) field[!eq]=value&field[exists]&field[!null]	**GET /api/v3/products?label[!eq]=Shirt&label[exists]** **GET /api/v3/products?label[!eq]=Shirt&label[!null]**
Field is not null	**field[!null]**	**GET /api/v3/products?label[!null]**
Field is null	**field[null]**	**GET /api/v3/products?label[null]**
Field = '' OR Field = []	field[len]=0	**GET /api/v3/products?label[len]=0** **GET /api/v3/products?overwritten_attributes[len]=0**
NOT (Field = '' OR Field = []) AND EXISTS(Field)	field[len][gt]=0	**GET /api/v3/products?label[len][gt]=0**
IS UNDEFINED (field not exists)	**field[!exists]**	**GET /api/v3/products?label[!exists]**
IS NOT UNDEFINED	**field[exists]**	**GET /api/v3/products?label[exists]**
Field like '%value%' (IGNORE CASE)	**field[contains:ignorecase]=value**	**GET /api/v3/products?sku[icontains:ignorecase]=shirt**
Field like '%value%' (CASE SENSITIVE)	**field[contains]=value**	**GET /api/v3/products?sku[icontains]=shirt**
With ARRAYs: ARRAY[field] && ARRAY[<value1>, <value2>] With jsonb arrays: '["a", "b", "c"]'::jsonb ?& array['a', 'b']	**field[intersects]=value** **field[!intersects]=value**	**GET /api/v3/products?overwritten_attributes[intersects]=color&overwritten_attributes[intersects]=size** **GET /api/v3/products?overwritten_attributes[!intersects]=color**
Field in ('value1', 'value2')	**field[in]=value1&field[in]=value2**	**GET /api/v3/products?status[in]=Draft&status[in]=Completed**
	_or[#]field=value1&_or[#]field=value2	**GET /api/v3/products?_or[0]status=Draft&_or[0]status=Completed**
Field not in ('value1', 'value2') OR field IS NULL	(If the field is undefinable) _or[#]field[!in]=value1&_or[#]field[!in]=value2&_or[#]field[!exists] (If the field is nullable) _or[#]field[!in]=value1&_or[#]field[!in]=value2&_or[#]field[!	**GET /api/v3/products?_or[0]status[!in]=Draft&_or[0]status[!in]=Completed&_or[0]status[!exists]**

Filters		
	null] (If the field is undefinable and nullable) _or[#]field[!in]=value1&_or[#]field[!in]=value2&_or[#]field[!exists]&_or[#]field[!null]	
`Field not in ('value1', 'value2')`	**field[!in]=value1&field[!in]=value2**	`GET /api/v3/products?status[!in]=Draft&status[!in]=Completed`
`Field > #`	**field[gt]=#**	`GET /api/v3/products?num_variations[gt]=1`
`Field >= #`	**field[gte]=#**	`GET /api/v3/products?num_variations[gte]=1`
`Field < #`	**field[lt]=#**	`GET /api/v3/products?num_variations[lt]=1`
`Field <= #`	**field[lte]=#**	`GET /api/v3/products?num_variations[lte]=1`
`Generic Length Comparison` `LEN(field) <# comparison> = value`	<field>[length][<eq|!eq|lt|lte|gt|gte>]=<value>	`GET /api/v3/products?label[length][!eq]=3`
`Field1 like '%value%' or Field2 like '%value%' (IGNORE CASE)`	_or[#]field1[contains:ignorecase]=value& _or[#]field2[contains:ignorecase]=value	`GET /api/v3/products?_or[0]sku[contains:ignorecase]=shirt&_or[0]label[contains:ignorecase]=shirt`
`field BETWEEN #1 AND #2`	field[gte]=#1&field[lte]=#2	`GET /api/v3/products?num_variations[gte]=1&num_variations[lte]=5`

`Generic Days passed Comparison`
 	
`UTCDATE(Field) <# comparison> (UTCDATE(NOW(timezone)) - DAYS(#))`
	<field>[<days_passed_function>][<eq|!eq|lt|lte|gt|gte>]=<value>	`GET /api/v3/products?created[days_passed][gte]=7`

Related entities filtering transversal			
	Current	Possible Alternative	

Basic entity transversal	Not possible	**related-entity.attribute[<filter>]=value**
		
		(x:m related entities can be named on a per relation basis, developers control if its named ""<plural_entity_name>" or "<singular_entity_name>_set")
			`GET /api/v3/products?categories.name=Shoes`
			
			`GET /api/v3/products?product_family.name=Apparel`
			
			`GET /api/v3/products?parent.sku=ABC`

Related entities filtering transversal			
Custom attributes	{"operator": "<filter>", "field": "attributes.name", "value": "value"}	**attributes.name[<filter>]=value**	**GET /api/v3/products?attributes.color=red** **GET /api/v3/products?attributes.weight[gt]=5**

Product relationships	"relationship_filters":[[{
	"product_ids":[{
	"id": "<product_id>",
	"qty_operator":
	"<op2>",
	"value": #
	}],
	"relationship_id":
	"<relationship_id>",
	"operator": "<op1>"
	}]],
		**product_relationships.relationship_id=<relationship_id>&
		product_relationships.related_products.related_product_id=<product_id>&
		product_relationships.related_products.quantity[<op2>]=#**
			GET /api/v3/products?product_relationships.relationship_id=<rid>&product_relationships.related_products.related_product_id=<pid>&product_relationships.related_products.quantity[gte]=2
			
			GET /api/v3/products?_fields=id&_fields=sku&_fields=assets.id
			
			GET /api/v3/products?_fields=id&_fields=sku&_fields=parent.id&_fields=parent.sku

	Eg1 [[{ "relationship_id":"5d89c6c696c8f4a322df4bbc", "operator":"exists", "product_ids":[], }]]	**Eg1 product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc**	**GET /api/v3/products?product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc**
	Eg2 [[{ "relationship_id":"5d89c6c696c8f4a322df4bbc", "operator":"!exists", "product_ids":[] }]]	Eg2 **_!exists[0]product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc** **For the public PIM API V3.0, it will only be allowed for relationships.id or relationships.label and without being grouped with _and or _or**	**GET /api/v3/products?_!exists[0]product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc**
		relationships[have]id=5d89c6c696c8f4a322df4bbc relationships[have]label[!eq]='pack' relationships.id[have]5d89c6c696c8f4a322df4bbc	
		_alias[relationships]=alias& .alias.id=5d89c6c696c8f4a322df4bbc& .alias[len]=0	
		_fields=relationships.id&	

Related entities filtering transversal			
		relationships[!contains]={"id": "5d89c6c696c8f4a322df4bbc"}	
		relationships[contains_kv:label:pack][eq]=false	

	Eg3
	[[{
	"product_ids":[{
	
	"id":"5d89c68596c8f4a322df4bb9"
	,
	
	"qty_operator":"exists",
	"value":null}],
	"relationship_id":
	"5d89c6c696c8f4a322df4bbc",
	"operator":"exists"
	}]]
		product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc&product_relationships.related_product_id=5d89c68596c8f4a322df4bb9	**GET /api/v3/products?product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc&product_relationships.related_products.related_product_id=5d89c68596c8f4a322df4bb9**

	Eg4
	[[{
	"product_ids": [{
	"id":
	"5d89c67796c8f4a322df4bb8",
	"qty_operator": "gt",
	"value": [2]}],
	"relationship_id":
	"5d89c6c696c8f4a322df4bbc",
	"operator": "exists"
	}]]
		product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc&product_relationships.related_product_id=5d89c67796c8f4a322df4bb8&product_relationships.related_products.quantity[gt]=2
		
		product_relationships
		Id_product | Id_relation | linked_to | quantity
		
		(Ambiguous: The link could belong to a different relationship)
		relationships.label=packed_together&
		relationships.links.product_id=5d89c67796c8f4a322df4bb8&
		relationships.links.quantity[gt]=2
		
		Aliases might solve this
			GET /api/v3/products?product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc&product_relationships.related_products.related_product_id=5d89c67796c8f4a322df4bb8&product_relationships.related_products.quantity[gt]=2

	Eg5
	[[{
	"product_ids": [{
		product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc&product_relationships.related_product_	**GET /api/v3/products?product_relationships.relationship_id=5d89c6c696c8f4a322df4bbc&product_relationships.related_products.related_product_id=5d89c67796c8f4a322**

Related entities filtering transversal			
	"id": "5d89c67796c8f4a322df4bb8", "qty_operator": "bte", "value": [1, 2]}], "relationship_id": "5d89c6c696c8f4a322df4bbc", "operator": "exists" }]]	**id=5d89c67796c8f4a322df4bb8& product_relationships.relrelated_products.quantity[lte]=2& product_relationships.relrelated_products.quantity[gte]=1** **(Ambiguous: The link could belong to a different relationship) relationships.id=5d89c6c696c8f4a322df4bbc& relationships.links.id=5d89c67796c8f4a322df4bb8& relationships.links.quantity[gte]=1& relationships.links.quantity[lte]=2** **Aliases might solve this**	df4bb8&product_relationships.related_products.quantity[gte]=1&product_relationships.related_products.quantity[lte]=2

Returned first level fields		
Selected ones	**_fields=<field1>&_fields<field2> (Optional)**	GET /api/v3/products?_fields=sku&_fields=status
All	**Default behaviour**	GET /api/v3/products
	_fields=*	GET /api/v3/products?_fields=* GET /api/v3/products?_fields=*&_fields=categories.name

Returned multilevel fields		
Selected ones	**_fields=<expansion1>.<field1>& [*...]&_fields=<expansion1>.<fieldn>& [*...]&_fields=<expansionn>.<field1>& [*...]&_fields=<expansionn>.<fieldn>** **If the <field> is *, then all fields are returned (default behaviour for ".")**	GET /api/v3/products?_fields=sku&_fields=product_family.name GET /api/v3/products?_fields=product_family.family_attributes.attribute.label GET /api/v3/products?_fields=attributes.color
All	_fields=<entity>.*	GET /api/v3/products?_fields=product_family.*

Related entities data expansion

_fields=<subentity>
_fields=<subentity>.<field|*>
	This will expand all related elements from that subentity to the parent one, regardless if any filters applied in the rest of the API call include that related entity too.
	
	As of now, this is the default and only approved behaviour.
		`GET /api/v3/products?_fields=product_family.*`
		
		`GET /api/v3/products?_fields=product_family.name`
		
		`GET /api/v3/products?_fields=product_family.family_attributes.*`

Name a relationship in `_fields` to resolve it. `_fields=*` and `GET` return the same result:

1. `_ids` fields hidden for now, as they expose denormalization (excluded from responses and for operation use)
2. `_id` fields are exposed as they are foreign keys correctly modeled
3. `_fields=related_entity` matches `_fields=related_entity.*` so it matches the behaviour of `_fields=json-field | _fields=json-field.*` and `GET /entity | GET /entity?_fields=*`
4. to-many relations (`` `assets` ``) are returned as a list. to-one relations (`` `thumbnail`, `parent` ``) are returned as a single object.

Pagination	
Possible Alternative	
_page=#&_page_size=#&_sort_by=<field>&_filtered_count=<bool>&_total_count=<bool>	`GET /api/v3/products?_page=2&_page_size=50&_sort_by=-sku&_sort_by=label` _page defaults to 1; _page_size to 25 (max 1000). _sort_by can appear multiple times (max XX), -field for descending; no default sort. _filtered_count and _total_count do not exist yet.

`_sort_by=<field>`
`<field>` can be any scalar field that:

1. Is a 1st level field of the parent entity.
2. Is the key at which you arrive traversing a 1st level dict of the parent entity.
3. Any of the 1 and 2 criteria, traversing related entities that are pointed to with a single direct FK.
4. Operators/functions like MAX, MIN, AVG, LENGTH
5. 1, 2 and 3 traversing related entities that are in the N part of a 1:N or N:N
6. Non scalar fields

Multiple _sort_by=<field> are allowed, the order in which they appear is the order of the sort.

Follow `pagination.next_page` until it is missing.

If a record is created, deleted or changes its sort value during the read, the records after it shift by one
position (That can skip or repeat one record at a page boundary. For large exports while the catalog is being edited, use keyset paging on `id`: keep `_page=1`, sort by `id`, and pass the last id received as `id[gt]`)

URL S STANDARDIZATION

	V3			
Action	HTTP VERB	PATH	Notes	

Search	GET	**api/v3/<plural_entity_name>[?<search_query>]**
		
		No `search_query` means "get all", unless there is an HTTP payload indicating its own filters
			Response:
			200
			{
			"data": [
			*<json_object_1>,
			*...,
			*<json_object_n>
			],
			"pagination": {
			*"next_page": <url>,
			*"previous_page": <url>,
			*"filtered_count": #,
			*"total_count": #
			}
			}
			4xx/5xx
			{
			"errors":[{
			"name": "<name>",
			"description": "<description>"
			}]
			}
				`GET /api/v3/products?status=Draft&_page_size=25 → 200`

Get one entity instance	GET	**api/v3/<plural_entity_name>/<identifier>?<*expansions>*&<fields_selection>**
		api/v3/<plural_entity_name>/<identifier><subentities>?<*expansions>*&<fields_selection>
		
		<subentities> = (/<subentity>)+
		<subentity> = <x:m related entity>|<x:1 related_entity>
		<x:m related entity> = <plural_entity_name>/<identifier>
		<x:1 related_entity> = <entity_name>
			Identifiers will usually be the primary key field of the entity.
			
			For some entities, the identifiers can be other fields. They require dto be fields that are unique for one account<>entity.
			
			Eg. For products, both id and sku (correctly url-encoded) can be used as identifiers.
				`GET /api/v3/products/{product_id} → 200`
				
				`GET /api/v3/products/{product_id}?_fields=sku → 200`
				
				`GET /api/v3/products/{product_id}/relationships/{relationship_id}`
				
				`GET /api/v3/products/{product_id}/relationships/{relationship_id}/related-products/{related_product_id}`
				
				`GET /api/v3/product-families/{product_family_id}/attributes/{attribute_id}`

Create	POST	**api/v3/<plural_entity_name>**	**“EMPTY” VALUES SUCH AS “” OR [] ARE STORED VERBATIM** **RETURN 201 { id: }**	`POST /api/v3/products {"sku": "SKU-1", "label": "Shirt"} → 201`

Override selected fields of one entity instance	PATCH	**api/v3/<plural_entity_name>/<identifier>**
		api/v3/<plural_entity_name>/<identifier><subentities>
			“EMPTY” VALUES SUCH AS “” OR [] ARE STORED VERBATIM
			
			RETURN 204 OK
				`PATCH /api/v3/products/{product_id} {"label": "New"} → 204`
				
				`PATCH /api/v3/products/{product_id}/attributes/color → 204`

Delete one entity instance	DELETE	**api/v3/<plural_entity_name>/<identifier>**
		api/v3/<plural_entity_name>/<identifier><subentities>
			RETURN 204 OK	`DELETE /api/v3/products/{product_id} → 204`
				
				`DELETE /api/v3/products/{product_id}/relationships/{relationship_id} → 204`

Get one entity instance field	GET	**api/v3/<plural_entity_name>/<identifier>/<field>**
		api/v3/<plural_entity_name>/<identifier><subentities>/<field>
				`GET /api/v3/products/{product_id}/label → 200 {data: "Shirt"}`
				
				`GET /api/v3/products/{product_id}/attributes/color`

Create an entity instance field	POST	**api/v3/<plural_entity_name>/<identifier>/<field>** **api/v3/<plural_entity_name>/<identifier><subentities>/<field>**	It will only work on attributes **“EMPTY” VALUES SUCH AS “” OR [] ARE STORED VERBATIM**	
Delete an entity instance field	DELETE	**api/v3/<plural_entity_name>/<identifier>/<field>** **api/v3/<plural_entity_name>/<identifier><subentities>/<field>**	It will only work on attributes	`DELETE /api/v3/products/{product_id}/attributes/color → 204`
Advanced updates	PATCH	**api/v3/<plural_entity_name>/<identifier>** **api/v3/<plural_entity_name>/<identifier><subentities>**	Indicated by adding the content type: application/json-patch+json https://datatracker.ietf.org/doc/html/rfc6901 https://datatracker.ietf.org/doc/html/rfc6902	

1:M Link a new entity instance	POST	**api/v3/<plural_entity_name>/<identifier>/<plural_related_entity_name>**	{
			"id": <related_entity_id>,
			[<M:M stuff>]
			}
				`POST /api/v3/assets/{asset_id}/categories {"id": "<asset_category_id>"} → 201`
				
				`POST /api/v3/product-attributes/{attribute_id}/groups {"id": "<group_id>"} → 201`
				
				`POST /api/v3/products/{product_id}/relationships {"relationship_id": "<rid>", "related_products": [{"product_id": "<pid>", "quantity": 1}]} → 201`
				
				`POST /api/v3/product-families/{product_family_id}/attributes {"attribute_id": "<aid>", "level": "no_level"} → 201`

M:M get a association entity instance	GET	**api/v3/<plural_entity_name>/<identifier>/<plural_related_entity_name>/<linked_id>**		`GET /api/v3/products/{product_id}/relationships/{relationship_id} → 200`
				
				`GET /api/v3/product-families/{product_family_id}/attribut`

				es/{attribute_id} → 200

M:M Edit a Relationship entity instance	PATCH	**api/v3/<plural_entity_name>/<identifier>/<plural_related_entity_name>/<linked_id>**	{
			"id": <related_entity_id>,
			[<M:M stuff>]
			}
				PATCH
				/api/v3/products/{product_id}/relationships/{relationship_id} {"related_products": [{"product_id": "<pid>", "quantity": 2}]} → 204
				
				PATCH
				/api/v3/product-families/{product_family_id}/attributes/{attribute_id} {"level": "variant_level"} → 204

1:M Unlink a new entity instance	DELETE	**api/v3/<plural_entity_name>/<identifier>/<plural_related_entity_name>/<linked_id>**	It deletes all the coincidences in the strange case of M:M relationships that allow repetitions. Unlinking something not linked → 404	DELETE /api/v3/assets/{asset_id}/categories/{asset_category_id} → 204 DELETE /api/v3/product-families/{product_family_id}/attributes/{attribute_id} → 204 DELETE /api/v3/products/{product_id}/relationships/{relationship_id} → 204

Bulk additions	POST	**api/v3/-/bulk/<plural_entity_name>**
		
		List of items equal to the non bulk POST
		Payload:
		{
		*"webhook": {"url": "<url>"},
		*"external_reference": <str>,
		"data": [{},{} ... {}]
		}
			Webhooks 1st implementation comes with absolutely no guaranties.
			
			external_reference is UNIQUE per account and action-type
			
			Response:
			{
			"bulk_job_id": <id>
			}	

Bulk edits (a filter and the same edit for all matches)	PATCH	**api/v3/-/bulk/<plural_entity_name>/?<search_query>**
		
		Same payload as the non bulk PATCH
		Payload:
		{
		*"webhook": <url>,
		*"external_reference": <str>,
		"data": **{"<field_1>": <value_1>, ..., "<field_n>": <value_n>}**
		}
			"EMPTY" VALUES SUCH AS "" OR [] ARE STORED VERBATIM AND DON'T MEAN DELETION	

Bulk edits (different edit per entity instance)	PATCH	**api/v3/-/bulk/<plural_entity_name>/** **List of items equal to the non bulk PATCH plus the id** **Payload:** { *"webhook": {"url": "<url>"},	**"EMPTY" VALUES SUCH AS "" OR [] ARE STORED VERBATIM AND DON'T MEAN DELETION**	

		***"external_reference": <str>,** **"data": [{"id": <id_1>, ...}, ..., {"id": <id_n>, ...}]** **}**		
Bulk deletes by filter	DELETE	**api/v3/-/bulk/<plural_entity_name>/?<search_query>** **{** ***"webhook": {"url": "<url>"},** ***"external_reference": <str>** **}**		
Bulk deletes by individual items	DELETE	**api/v3/-/bulk/<plural_entity_name>/** **Payload:** **{** ***"webhook": {"url": "<url>"},** ***"external_reference": <str>,** **"data": [id, id, id]** **}**		
Advanced bulk updates	PATCH	**api/v3/-/bulk/-/edit/<plural_entity_name>/?<search_query>** **The payload format is the same one of the advanced updates**		
Bulk Individual updates	POST	**api/v3/-/bulk/-/edit** **The payload contains a list of individual actions (deletions, creations and updates)** **{** ***"webhook": {"url": "<url>"},** ***"external_reference": <str>,** **"data": [{ "entity: 'products", "action": "PATCH|DELETE|POST", data: { individual-object }}]** **}** **The result, except for performance, is equivalent to individually calling the other update endpoints one by one.**		
Bulks: check job	GET	**api/v3/-/bulk/-/jobs/<bulk_job_id>**	**Response:** **{** **"state": <str>,** **"webhook": <object|none>,**	

			"external_reference": <str|none>, **"created": <datetime>,** **"updated": <datetime>,** **"data": <object|none>** **}**	
Bulks: search job	GET	**api/v3/-/bulk/-/jobs/?<search_query>** **queryable attributes:** **external_reference <str>** **state <str>**		
Bulks: transition job	POST	**api/v3/-/bulk/-/jobs/<bulk_job_id>/<cancel|pause|resume|retry>** **api/v3/-/bulk/-/jobs/<bulk_job_id>/-/<cancel|pause|resume|retry>**		

Ad-Hoc async action (eg. Shopify export)	POST	**api/v3/-/<action-name>/**
		
		{
		***"webhook": {"url": "<url>"},**
		***"external_reference": <str>,**
		"data": "<whatever>"
		}
		
		eg. api/v3/-/shopify-export/
			Webhooks 1st implementation comes with absolutely no guaranties
			
			external_reference is UNIQUE per account and action-type
			
			Response:
			{
			"action-name_job_id": <id>
			}
			
			eg.
			{
			"shopify-export_job_id": <id>
			}	

Ad-Hoc async action: check job	GET	**api/v3/-/<action-name>/-/jobs/<action-name_job_id>**	**Response:** **{** **"state": <str>,** **"webhook": <object|none>,** **"external_reference": <str|none>,** **"created": <datetime>,** **"updated": <datetime>,** **"data": <object|none>** **}**	
Ad-Hoc async action: search jobs	GET	**GET** **api/v3/-/<action-name>/-/jobs/?<search_query>**		

		queryable attributes: **external_reference <str>**		
Ad-Hoc async action: transition job	POST	**api/v3/-/bulk/-/<action-name>/<action-name_job_id>/<cancel|pause|resume|retry>** **api/v3/-/<action-name>/-/jobs/<action-name_job_id>/-/<cancel|pause|resume|retry>**		

200 OK → SYNC
201 OK → SYNC → ID
202 JOB -> ASYNC

Response Conventions for Entity Modification Endpoints

General

For the definition of the new API, the following conventions will apply:

- **POST** requests for creating entities, **PATCH** requests for updating entities, and **PATCH** requests will **not** return the modified entity in the response (204 - No Content).
- Instead:
    - A **201 Created** status will be returned if the creation or modification has been successfully completed.

      `201 Created`

      The request succeeded, and a new resource was created as a result. This is typically the response sent after `POST` requests, or some `PUT` requests.

    - A **202 Accepted** status will be returned if the request has been accepted but the processing is still ongoing.

      `202 Accepted`

      The request has been received but not yet acted upon. It is noncommittal, since there is no way in HTTP to later send an asynchronous response indicating the outcome of the request. It is intended for cases where another process or server handles the request, or for batch processing.

- In the case of **201 Created**, the response body will include the **ID of the created or modified entity**.

- In the case of **202 Accepted**, the response body will include a **process_id** that can be used to track the status of the ongoing operation.

PATCH / 204

Instead, the recommended approach is to use **PATCH** requests, where only the intended changes are explicitly specified. This method provides more precise control over updates, reduces the risk of accidental data loss, and simplifies concurrency management. Adopting PATCH as the standard for modifications ensures a safer and more maintainable API design.

DELETE / 204

In API V3 a `DELETE` operation erases the entity or field (if using a path based `DELETE`) it is performed over, it doesn't set null or applies defaults (it can cause inheritance to resolve for the field)

ANEXO

Carácter	Significado en URL
=	separa clave/valor
&	separa parámetros
?	inicia la query
#	inicia fragmento
+	puede interpretarse como espacio
,	a veces reservado por proxies antiguos
< >	no válidos en URL (se deben escapar)
"	pueden romper encoding
%	indica secuencia de escape
( )	válidos pero deberían codificarse según RFC 3986

(espacio)	se convierte en + o %20

PROCESS MANAGER API

URL S STANDARDIZATION

	V3		
Action	HTTP VERB	PATH	Notes
Search, get, patch, post, delete	ALL	**ALL CRUD OPERATIONS EXPOSED WITH API V3**	For processes, we won't let PATCHes for their `statuses` (process transitions will be handled in ad-hoc endpoints)

	POST	**/api/vx/<to_be_decided>/**	INPUT PAYLOAD:
			{
			"external_id": <optional>,
			"webhook": <optional>,
			“payload”: {
			process_type:
			…
			input_data:
			}
			RESPONSE:
			{
			"
			}